

Akash Sarode:
M-SB-Q-Stud — Final Notebook



General Overview of Responsibilities

Backend Engineering

- Designed and implemented the **secure file upload system**, ensuring robust validation for PDF processing.
- Developed the **SQLite database initialization script** to automate schema creation and data seeding.
- Built robust **RESTful API endpoints** to act as the integration layer between frontend and backend.

Scribe and Documentation

- Authored comprehensive notes for **8 team meetings**, creating a persistent project record.
- Synthesized complex technical discussions into **actionable summaries** to align the engineering team.
- Managed **action items and attendance** to ensure accountability and drive weekly progress.

Frontend Development

- Developed the **Question Database interface**, creating a structured layout to organize and display generated questions, answers, and metadata.
- Refined the **visual hierarchy and styling**, cleaning up the UI components to ensure complex question data is readable and accessible.
- Integrated the display logic with backend APIs to enable **dynamic data retrieval**, allowing users to seamlessly browse and interact with the question database page.

Commits on Nov 10, 2025

Fixed small error

 Akash Sarode committed 3 weeks ago

Fixed api error

 Akash Sarode committed 3 weeks ago

Added frontend integration

 Akash Sarode committed 3 weeks ago

Implement SQL Lite db

 Akash Sarode committed 3 weeks ago

Commits on Nov 3, 2025

Added file upload functionality

 Akash Sarode committed on Nov 3

Updated Project Documentation

 Akash Sarode committed on Nov 3



Scribe Role Contributions

Project Documentation and Knowledge Management

- **Authored comprehensive meeting minutes** for a total of **8 weekly team meetings**, ensuring a persistent and retrievable record of the project's evolution.
- **Synthesized complex technical discussions** into concise "Meeting Summaries," distilling raw brainstorming sessions into clear, legible information regarding architectural decisions and feature planning.
- **Standardized the documentation format**, establishing a consistent structure that included Agendas, Attendees, and Key Discussions to maintain organizational clarity throughout the development lifecycle.

Agile Process and Action Item Tracking

- **Managed the administrative workflow** by defining and recording "Action Items and Next Steps," ensuring that high-level concepts discussed in meetings were converted into concrete tasks.
- **Facilitated team accountability** by tracking attendance and agenda items, ensuring all team members (Team Lead, GitHub Updater, etc.) were aligned on their specific responsibilities for the upcoming sprint.
- **Bridged communication gaps** by acting as the central information hub, allowing team members to review past decisions and "Lessons Learned" without relying on memory alone.

Project: M-SB-Q-Stud Weekly Meeting Scribe Notes

Date: October 23, 2025

Attendees: Romir Varshney, Kevin Kim, Rohan Konanki, Akash Sarode

Scribe: Akash Sarode

Agenda

- Discussion on Application Deployment
- PACE Cluster Demonstration
- Action Items and Next Steps

Meeting Summary & Key Discussions

The team met to discuss critical infrastructure and feature planning for the M-SB-Q-Stud project. The main topics included deployment on the PACE Cluster, user authentication methods, and a hands on demonstration of the PACE environment.

1. Deployment Strategy: PACE Cluster

The team discussed using the Georgia Tech PACE Cluster as the primary environment for deploying the application. The conversation focused on the benefits of using its powerful computing resources, which will be essential for running the RAG model efficiently.

Key discussion points included:

- The necessary steps to prepare the application for deployment on the cluster.
- How to manage software dependencies and create a stable environment.
- The technical requirements for making the application web-accessible to students and TAs from the cluster.

2. User Authentication and Personalization

A significant portion of the meeting was dedicated to user authentication. The team agreed that allowing users to log in is crucial for personalizing the experience.

Meeting Notes Example



Frontend Contributions

Question Card

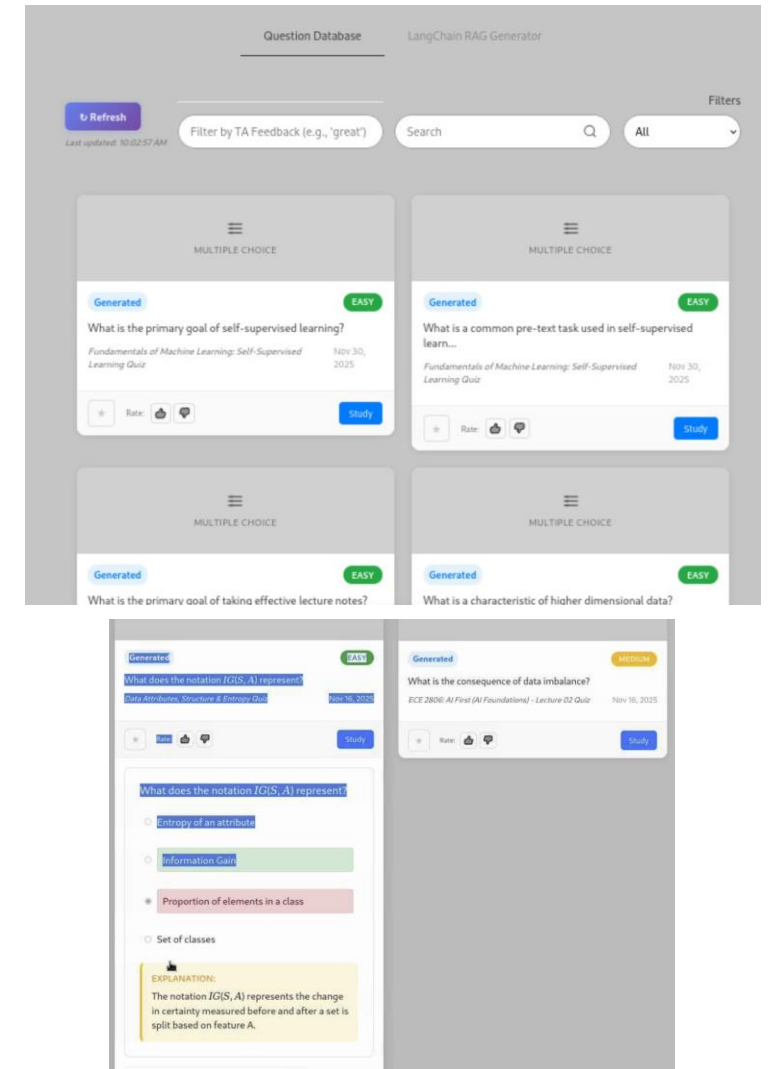
- Developed a **modular card component** that dynamically adapts its layout to support both Multiple Choice (radio buttons) and Short Answer (text input) formats.
- Visualized **metadata tags** (Course, Topic, Difficulty) with distinct color-coding, allowing students to quickly scan the database and identify relevant study material.
- Implemented an **interactive "Show Answer" toggle** that keeps the correct response and detailed explanation hidden by default to facilitate active recall.

Question Search

- Built a **robust filtering engine** that allows users to query the database by Course Name, Specific Topic, or Question Difficulty level.
- Implemented **search functionality** that instantly updates the question grid as users type, leveraging efficient state management for a responsive experience.

Question Success Rate / TA Feedback

- Integrated a **visual metrics display** to show accuracy statistics for each question, helping users identify high-difficulty concepts at a glance.
- Created a **dedicated UI section** for submitting qualitative feedback (TA Feedback), providing data that can later be used to improve question generation.
- Styled the **feedback components** to visually stand out from the main question text, ensuring that critical performance insights and tips capture the user's attention.



Backend Contributions

Database and Infrastructure

- Designed and implemented the initial **SQLite database schema**, establishing the necessary tables and relationships for the application.
- Wrote and tested a **database initialization script** that automates table creation and seeds initial data, ensuring consistent setup across development environments.
- Configured the database connection logic to handle distinct environments (development vs. testing).

File Management System

- Engineered the **server-side file upload logic**, allowing users to securely transmit documents to the backend.
- Implemented **input validation** middleware to restrict file types and sizes, preventing system errors and ensuring data integrity.
- Created a local storage strategy to organize uploaded files efficiently on the server file system.

API Development and Integration

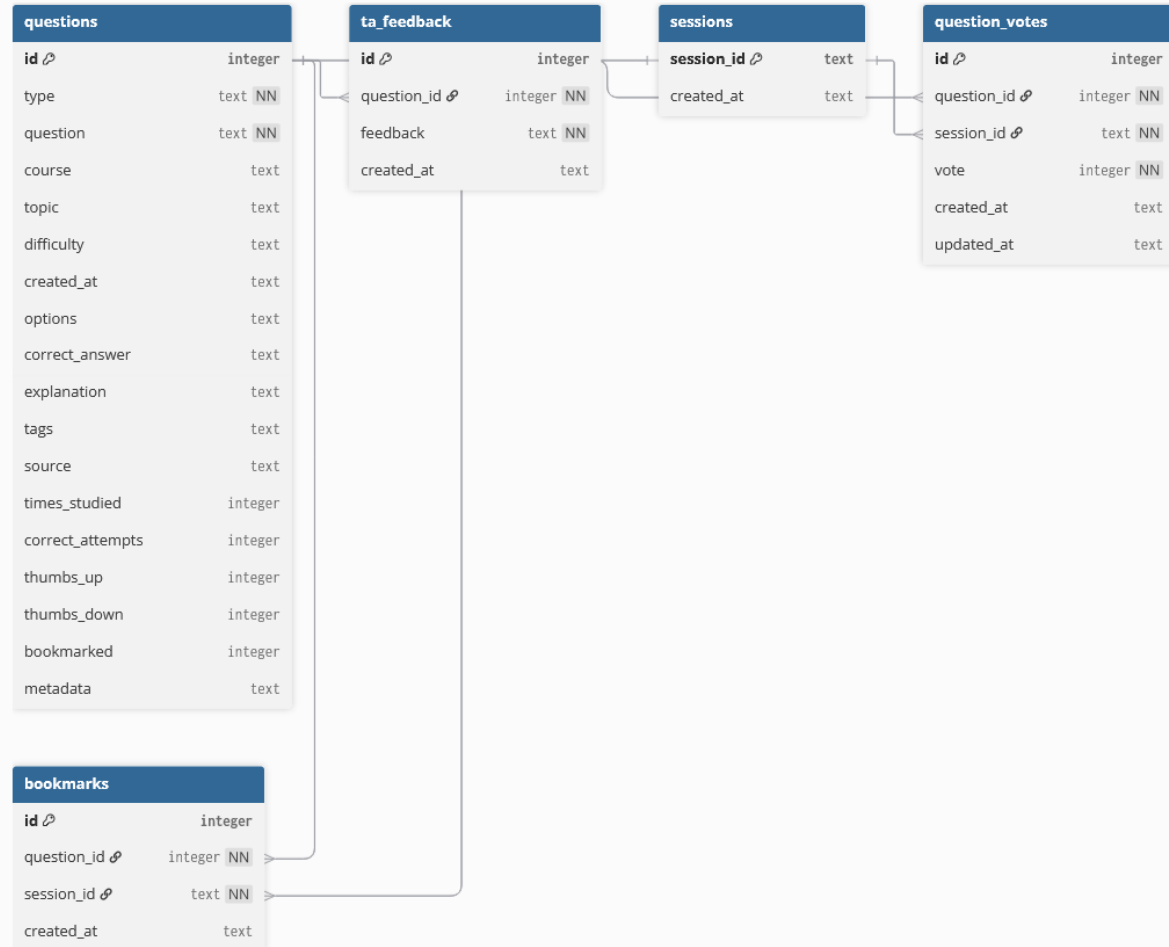
- Developed RESTful **API endpoints** to handle CRUD operations, bridging the gap between the SQLite database and the user interface.
- Structured API responses in standardized JSON format to ensure seamless parsing by the frontend team.
- Collaborated on **frontend integration**, debugging network requests to ensure successful data binding and state management.

```
-- Main questions table
CREATE TABLE IF NOT EXISTS questions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  type TEXT NOT NULL,
  question TEXT NOT NULL,
  course TEXT,
  topic TEXT,
  difficulty TEXT,
  created_at TEXT DEFAULT (strftime('%Y-%m-%dT%H:%M:%FZ', 'now')),
  options TEXT,
  correct_answer TEXT,
  explanation TEXT,
  tags TEXT,
  source TEXT,
  times_studied INTEGER DEFAULT 0,
  correct_attempts INTEGER DEFAULT 0,
  thumbs_up INTEGER DEFAULT 0,
  thumbs_down INTEGER DEFAULT 0,
  bookmarked INTEGER DEFAULT 0,
  metadata TEXT
);
```

The screenshot shows the 'LangChain RAG Quiz Generator' web application. At the top, there are tabs for 'Question Database' and 'LangChain RAG Generator'. Below the tabs, the title 'LangChain RAG Quiz Generator' is displayed, followed by the subtitle 'Generate quiz questions'. The main content area is divided into two sections. The first section, 'Upload PDF Document', prompts the user to 'Select a PDF file to use as context for quiz generation' and includes a 'Choose PDF file...' button and an 'Upload' button. The second section, 'Quiz Configuration', features a 'Total Questions: 5' input field. Below this, there are four rows of configuration options: 'Easy' (2), 'Medium' (2), 'Hard' (1), and 'Short Answer' (1). A note at the bottom of this section states 'Total is calculated from Easy + Medium + Hard. Remaining questions will be MCQ'. A 'Generate Quiz' button is located at the bottom right of the configuration section.



Database Diagram



dbdiagram.io

- **Custom SQL Design:** A comprehensive look at the database architecture created specifically for this application.
- **Scalability and Growth:** The database includes tables for advanced features, such as "bookmarks," that are not yet live in the frontend. In the future, we can fully implement these new features to enhance the project.
- **AI Content Repository:** Serves as the central persistent storage for all unique questions generated by the quiz model, ensuring that dynamic content is saved and accessible for other students
- **Feedback Loop Integration:** Stores user interactions (votes, bookmarks) alongside AI-generated questions, allowing the system to track which generated content is most effective for learning.



Challenges Faced

Schema Design for Unstructured AI Outputs

- **The Challenge:** The LLM (Llama 3.1) occasionally produced variable JSON structures for generated questions, which conflicted with the rigid data types required by the SQLite database.
- **The Solution:** I had to iteratively refine the database schema and implement strict validation logic in the API layer to ensure that diverse question formats (Multiple Choice vs. Short Answer) could be normalized and stored without errors.

Cross-Environment Path Resolution

- **The Challenge:** The file upload system faced critical "File Not Found" errors when moving between local development environments and the team's shared repository structure.
- **The Solution:** I engineered a robust path-handling utility in the backend that dynamically resolves relative paths, ensuring the RAG pipeline could consistently locate and chunk uploaded PDFs regardless of the operating system or directory structure.

Asynchronous State Synchronization

- **The Challenge:** The Question Database UI needed to display new questions immediately after generation, but the RAG process was computationally heavy and slow, causing UI desync.
- **The Solution:** I implemented efficient state management and asynchronous data fetching in the frontend, ensuring the "Database View" would automatically refresh and render the new content the moment the backend process completed.

Akash Sarode Fixed small error 803cf60 · 3 weeks ago 22 Commits		
anyreads	update	2 months ago
backend	Fixed small error	3 weeks ago
chroma_db	stuff	2 months ago
class stuff	update	2 months ago
myenv	Basic database in the form of a json file	2 months ago
public	random stuff	3 months ago
src	Fixed small error	3 weeks ago
.env	Fixed small error	3 weeks ago
.gitignore	update	2 months ago
.ollama.pid	Fixed small error	3 weeks ago
AIFirst_Lect02_Data-Attr-Struc-Entropy.pdf	Basic database in the form of a json file	2 months ago
README.md	Updated Project Documentation	last month
index.html	update	2 months ago
lecture_notes_02.pdf	Create lecture_notes_02.pdf	3 months ago
package-lock.json	random stuff	3 months ago
package.json	random stuff	3 months ago
questions.txt	Implemented basic frontend	3 months ago
requirements.txt	update	2 months ago



Reflection and My Personal Future Roadmap

Personal Reflection

- **From Prototype to Production:** Building the initial SQLite and file upload foundation highlighted the critical importance of scalability; I now understand why "quick start" solutions eventually need robust migration strategies.
- **The Power of Documentation:** Acting as Scribe revealed that clear technical specs are just as vital as code; the ability to translate architectural decisions into documentation saved us countless hours of realignment.

My Future Roadmap

- **Interactive Quiz Refinement:** Even though the roadmap in the final presentation didn't mention this, I personally think adding a **text prompting interface** on top of the file upload is an important feature to add. This would allow users to iteratively refine the output (e.g., "Make these harder," "Focus on Chapter 2") using natural language without needing to re-upload documents.
- **Frontend Component Architecture:** I aim to evolve my **Question Card** design into a universal "Quiz Engine" component, capable of dynamically rendering complex inputs like code blocks and matching grids based on a flexible JSON schema.
- **AI-Integrated Feedback Loops:** I intend to build the API and UI infrastructure for **TA Feedback**, creating the data pipelines necessary to capture user feedback provided from the question database and feed them back to the RAG model for continuous improvement.

9 of 9

[Akash Final Notebook] | [M-SB-Q-Stud]

The logo for AI Makerspace Nexus, featuring a central green circle with 'AI Makerspace Nexus' text, surrounded by various icons representing technology, education, and innovation.

The Georgia Institute of Technology logo, consisting of the letters 'GT' in a stylized font followed by the words 'Georgia Tech'.

Personal Reflection

- **From Prototype to Production:** Building the initial SQLite and file upload foundation highlighted the critical importance of scalability; I now understand why "quick start" solutions eventually need robust migration strategies.
- **The Power of Documentation:** Acting as Scribe revealed that clear technical specs are just as vital as code; the ability to translate architectural decisions into documentation saved us countless hours of realignment.

My Future Roadmap

- **Interactive Quiz Refinement:** Even though the roadmap in the final presentation didn't mention this, I personally think adding a **text prompting interface** on top of the file upload is an important feature to add. This would allow users to iteratively refine the output (e.g., "Make these harder," "Focus on Chapter 2") using natural language without needing to re-upload documents.
- **Frontend Component Architecture:** I aim to evolve my **Question Card** design into a universal "Quiz Engine" component, capable of dynamically rendering complex inputs like code blocks and matching grids based on a flexible JSON schema.
- **AI-Integrated Feedback Loops:** I intend to build the API and UI infrastructure for **TA Feedback**, creating the data pipelines necessary to capture user feedback provided from the question database and feed them back to the RAG model for continuous improvement.

